NXT WAVE™

WELCOME

NXT WAVE™

C++ STL : Deque & Stack

Agenda

C++ Bridge Course

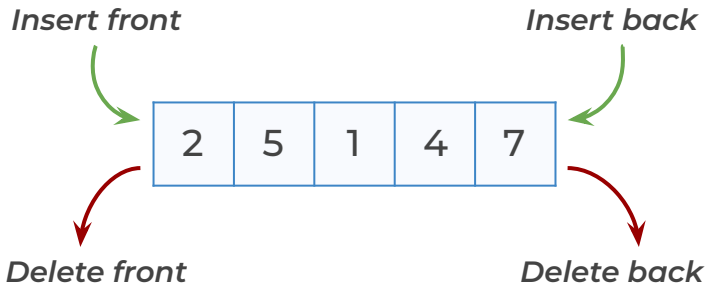
Container : Deque

Deque

Introduction

- *A deque (**double-ended queue**) is a dynamic data structure that allows efficient **insertion** and **deletion** from **both ends**—the **front** and the **back**.*

- *Both C++ **deque** and Python **collections.deque** are **double-ended queues**, meaning they **allow fast insertions** and **deletions** at both the **front** and **back**.*



Deque

Declaration and Initialization

C++

```cpp
deque<int> d1 = {1, 3, 5, 7, 9};
deque<int> d2(5, 10);
deque<int> d3(d1);
```

- *Initializing with values.*

Python

```python
d1 = deque([1, 3, 5, 7, 9])
d2 = deque([10] * 5)
d3 = deque(d1)
```

- *Initializing with values.*

Declaration and Initialization

C++

```
deque<int> d1 = {1, 3, 5, 7, 9};
deque<int> d2(5, 10);
deque<int> d3(d1);
```

- *Initializing with values.*
- *Creates a deque of size 5, all elements set to 10.*

Python

```
d1 = deque([1, 3, 5, 7, 9])
d2 = deque([10] * 5)
d3 = deque(d1)
```

- *Initializing with values.*
- *Creates a deque of size 5, all elements set to 10.*

Declaration and Initialization

C++

```
deque<int> d1 = {1, 3, 5, 7, 9};
deque<int> d2(5, 10);
deque<int> d3(d1);
```

- *Initializing with values.*
- *Creates a deque of size 5, all elements set to 10.*
- *Copying to another deque.*

Python

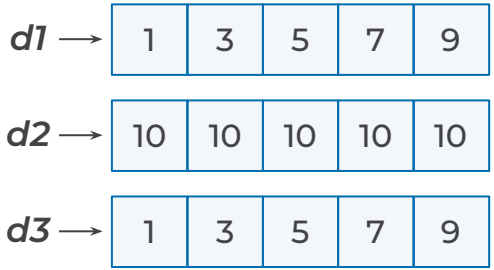
```
d1 = deque([1, 3, 5, 7, 9])
d2 = deque([10] * 5)
d3 = deque(d1)
```

- *Initializing with values.*
- *Creates a deque of size 5, all elements set to 10.*
- *Copying to another deque.*

Declaration and Initialization

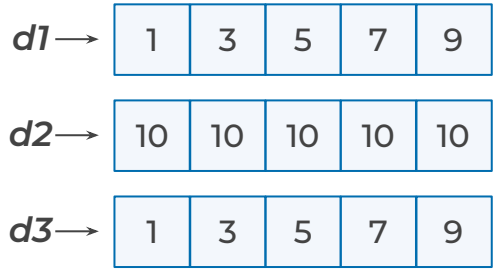
C++

```cpp
deque<int> d1 = {1, 3, 5, 7, 9};
deque<int> d2(5, 10);
deque<int> d3(d1);
```



Python

```python
d1 = deque([1, 3, 5, 7, 9])
d2 = deque([10] * 5)
d3 = deque(d1)
```

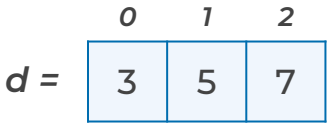


Deque

Adding Elements

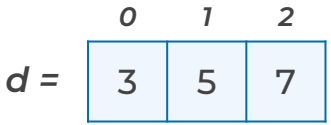
C++

```
deque<int> d = {3, 5, 7};
```



Python

```
d = deque([3, 5, 7])
```

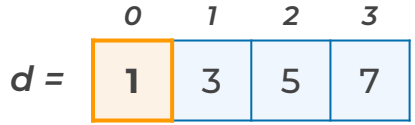


Adding Elements at the Front

Adds 1 at the front.

C++

```
deque<int> d = {3, 5, 7};
d.push_front(1);
```



Adds 1 at the front.

Python

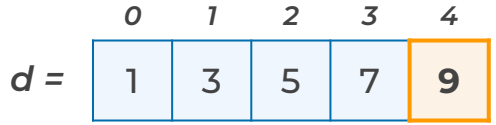
```
d = deque([3, 5, 7])
d.appendleft(1)
```

Adding Elements at the Back

Adds 9 at the back.

C++

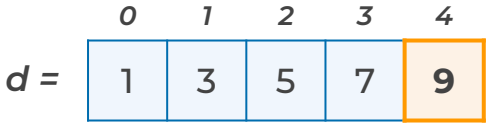
```
deque<int> d = {3, 5, 7};
d.push_front(1);
d.push_back(9);
```



Adds 9 at the back.

Python

```
d = deque([3, 5, 7])
d.appendleft(1)
d.append(9)
```



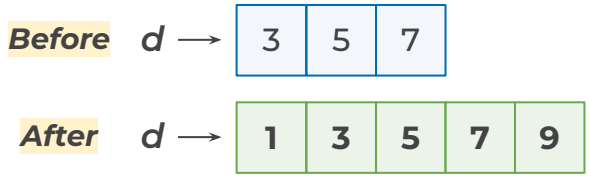
Deque

Adding Elements

Adding Elements

C++

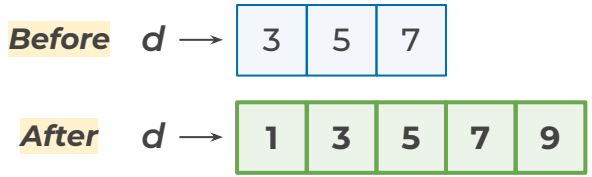
```
deque<int> d = {3, 5, 7};
d.push_front(1);
d.push_back(9);
```



Adding Elements

Python

```
d = deque([3, 5, 7])
d.appendleft(1)
d.append(9)
```

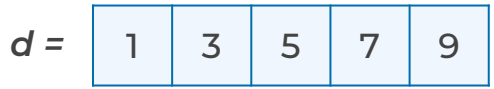


NXT WAVE™

Removing Elements

C++

```cpp
deque<int> d = {1, 3, 5, 7, 9};
```



Python

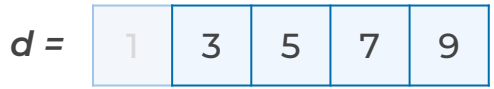
```python
d = deque([1, 3, 5, 7, 9])
```

Removing Elements from the Front

Removes first element (1).

C++

```
deque<int> d = {1, 3, 5, 7, 9};

d.pop_front();
```



Removes first element (1).

Python

```
d = deque([1, 3, 5, 7, 9])

d.popleft()
```

Removing Elements from the Back

Removes last element (9).

C++

```
deque<int> d = {1, 3, 5, 7, 9};

d.pop_front();
d.pop_back();
```

Removes last element (9).

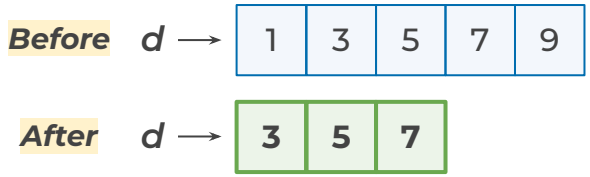
Python

```
d = deque([1, 3, 5, 7, 9])

d.popleft()
d.pop()
```

Removing Elements from the Front and Back

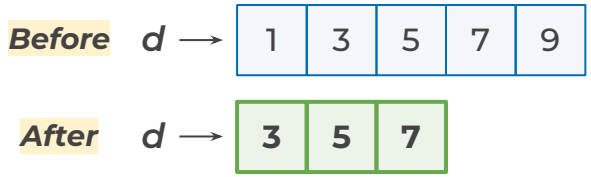
C++

```cpp
deque<int> d = {1, 3, 5, 7, 9};

d.pop_front();
d.pop_back();
```



Python

```python
d = deque([1, 3, 5, 7, 9])

d.popleft()
d.pop()
```



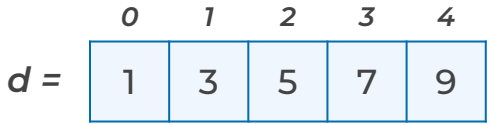
Accessing Front and Back Element

C++ deque supports front() and back() for accessing the first and last elements of a deque.

C++

```
deque<int> d = {1, 3, 5, 7, 9};
```

Output



Accessing Front and Back Element

C++ deque supports front() and back() for accessing the first and last elements of a deque.

C++

```
deque<int> d = {1, 3, 5, 7, 9};
cout << d.front();
```

Output

```
1
```

Accessing Front and Back Element

C++ deque supports front() and back() for accessing the first and last elements of a deque.

C++

```
deque<int> d = {1, 3, 5, 7, 9};
cout << d.front();
cout << d.back();
```

Output

```
9
```

Getting the Size

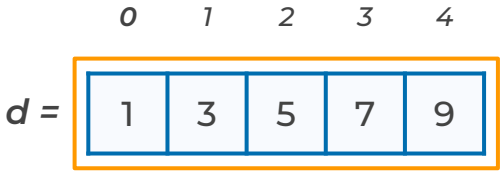
C++ deque supports size() for retrieving the number of elements present in the deque.

C++

```
deque<int> d = {1, 3, 5, 7, 9};
cout << d.size();
```

Output

5



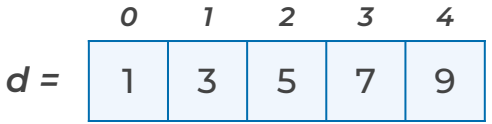
Iterating Over a Deque

C++

```
deque<int> d = {1, 3, 5, 7, 9};
```

*Deque of integers (**d**) initialized with*

*{**1**, **3**, **5**, **7**, **9**}*



Iterating Over a Deque

C++

```
deque<int> d = {1, 3, 5, 7, 9};
for (int x : d) {
  cout << x << " ";
}
```

Iterating using

Range-Based For Loop.

Output

```
1 3 5 7 9
```

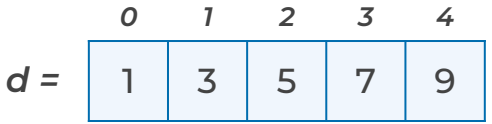
Reversing a Deque

C++

```
deque<int> d = {1, 3, 5, 7, 9};
```

*Deque of integers (**d**) initialized with*

*{**1**, **3**, **5**, **7**, **9**}*

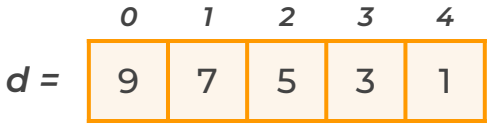


Reversing a Deque

C++

```
deque<int> d = {1, 3, 5, 7, 9};
reverse(d.begin(), d.end());
```

Reverses the deque



Deque

Summary

- ***Definition:***

  A double-ended queue (deque) allows fast insertion and deletion from both front and back.
- ***Adding Elements:***
  - **push_front(x):** Adds x at the front.
  - **push_back(x):** Adds x at the back.
- ***Removing Elements:***
  - **pop_front():** Removes the front element.
  - **pop_back():** Removes the back element.

Deque

Summary

- ***Accessing Front & Back Elements:***
  - **front():** Returns the first element.
  - **back():** Returns the last element.
- ***Inserting Elements at a Specific Position:***
  - **insert(it, x):** Inserts x at the given iterator position.
- ***Removing Elements at a Specific Position:***
  - **erase(it):** Removes the element at the given iterator position.

Deque

Summary

- ***Removing Elements by Value:***
  - **erase(remove(d.begin(), d.end(), x), d.end()):** Removes all occurrences of **x**.
- ***Iterating Over a Deque:***
  - Use **iterators** or **range-based for loops** for traversal.
- ***Reversing a Deque:***
  - **reverse(d.begin(), d.end()):** Reverses the order of elements.

Quiz Time!

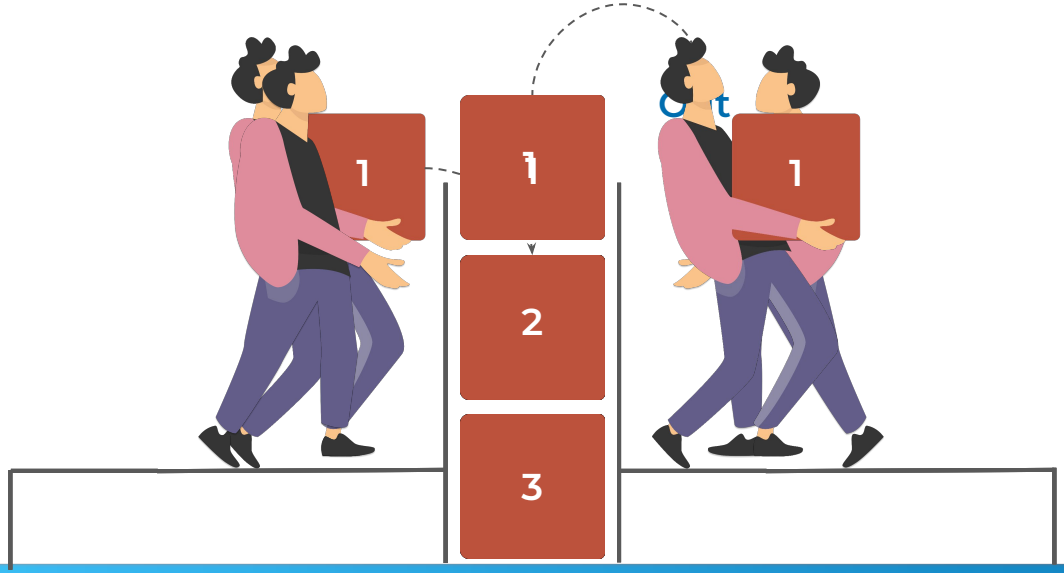
Container : Stack

Stack

Introduction

A stack is a **linear** data structure that follows the **LIFO** (**Last In**, **First Out**) principle.

This means the **last element inserted** is the **first one** to be **removed**.

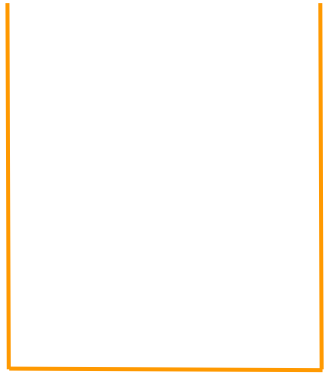


Declaration and Initialization

Creates an empty stack.

C++

```
stack<int> s;
```

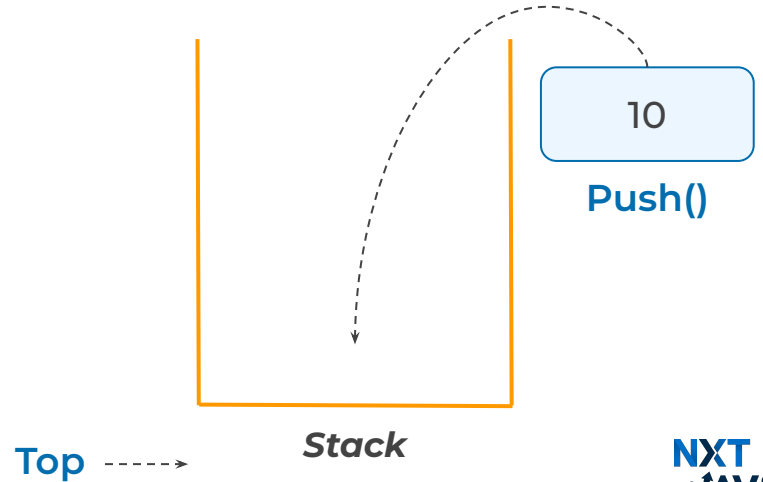


Pushing Elements onto the Stack

***Push 10** onto the stack.*

C++

```
stack<int> s;
s.push(10);
```

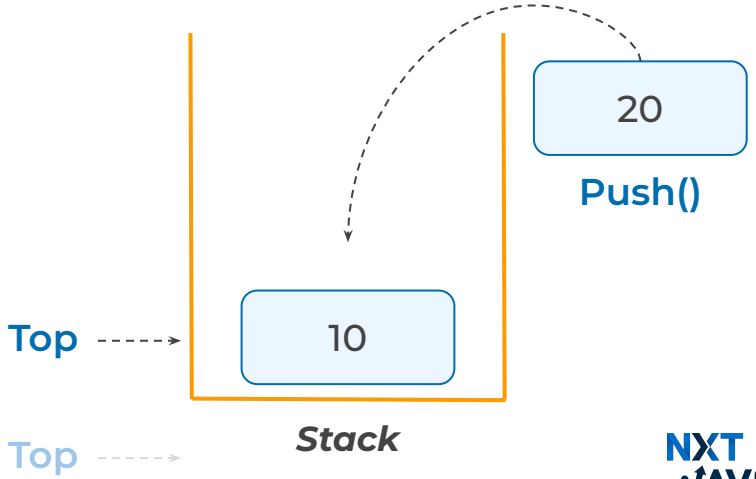


Pushing Elements onto the Stack

***Push 20** onto the stack.*

C++

```
stack<int> s;
s.push(10);
s.push(20);
```

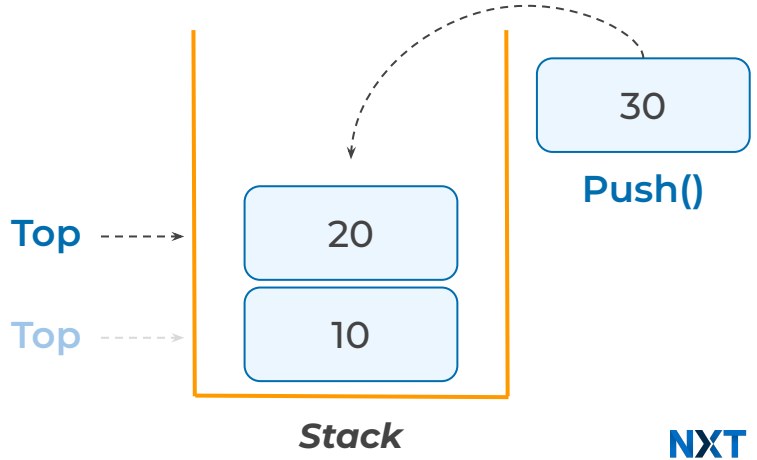


Pushing Elements onto the Stack

***Push 30** onto the stack.*

C++

```
stack<int> s;
s.push(10);
s.push(20);
s.push(30);
```

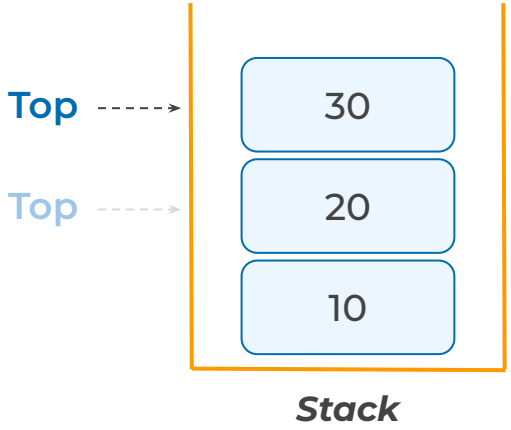


Pushing Elements onto the Stack

Stack Contents After Pushes.

C++

```
stack<int> s;
s.push(10);
s.push(20);
s.push(30);
```

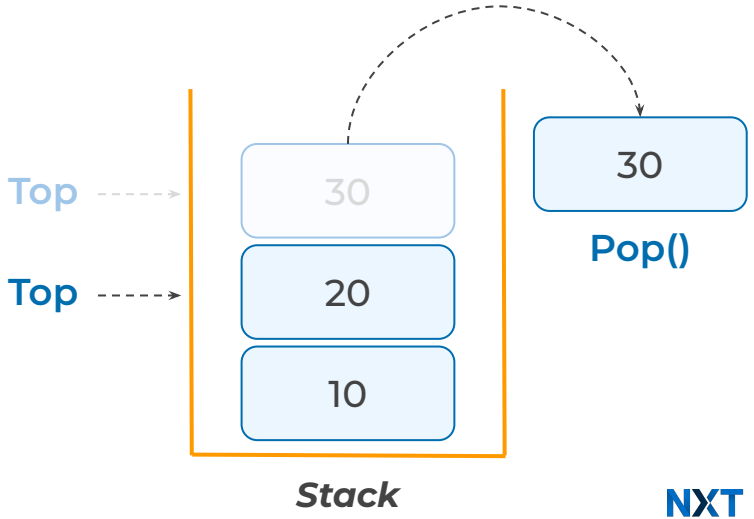


Popping Elements from the Stack

***Removes** the **top** element (**30**).*

C++

```
stack<int> s;
s.push(10);
s.push(20);
s.push(30);
s.pop();
```

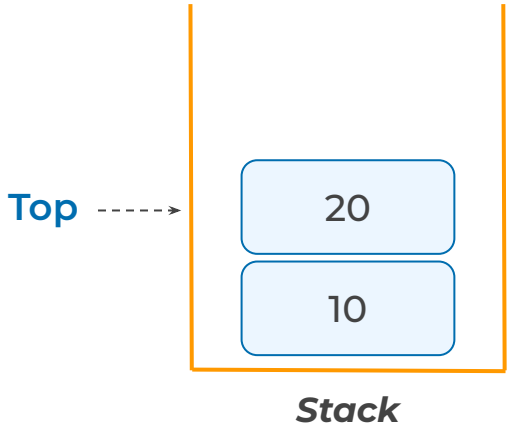


Popping Elements from the Stack

Stack Contents After Pop:

C++

```
stack<int> s;
s.push(10);
s.push(20);
s.push(30);
s.pop();
```



Getting the Top Element

Outputs 20 (Top element).

C++

```
stack<int> s;
.
.
.
cout << s.top();
```

Output

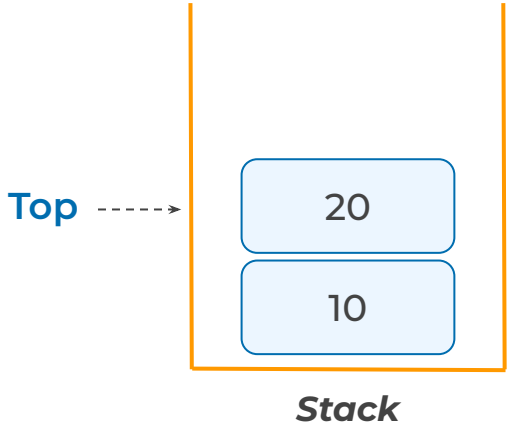
```
20
```

Checking if the Stack is Empty

s.empty() - returns 1 if stack is empty, otherwise returns 0

C++

```
stack<int> s;
.
.
.
if (s.empty()) {
 cout << "Stack is empty";
}
else {
  cout << "Stack is not empty";
}
```



Checking if the Stack is Empty

Output: Stack is not empty.

C++

```
stack<int> s;
.
.
.
if (s.empty()) {
 cout << "Stack is empty";
}
else {
  cout << "Stack is not empty";
}
```

Output

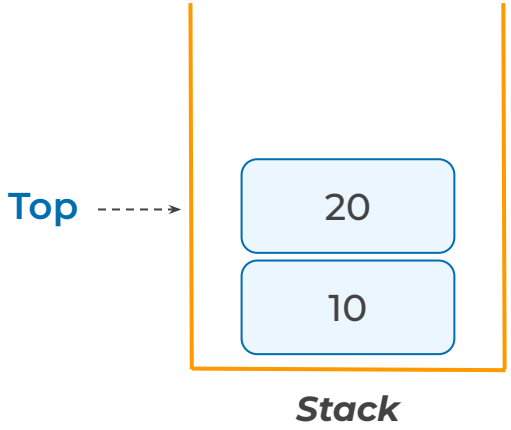
Stack is not empty

Checking if the Stack is Empty

`s.size()` ***returns number of elements in the stack.***

C++

```
stack<int> s;
.
.
.
cout << s.size();
```



Checking If the Stack is Empty

`s.size()` ***returns number of elements in the stack.***

C++

```
stack<int> s;
.
.
.
cout << s.size();
```

Output

```
2
```

Stack

Summary

- ***Definition***
  - A stack is a **LIFO (Last In, First Out)** data structure where the **last inserted element** is the **first** to be **removed.**
- ***Declaration & Initialization:***
  - **stack<int> s;** → Creates an empty stack**.**
- ***Pushing Elements onto the Stack:***
  - **push(x):** Adds x to the top of the stack.

Summary

- ***Popping Elements from the Stack:***
  - **pop():** Removes the top element from the stack.
- ***Getting the Top Element:***
  - **top()**: Returns the topmost element without removing it
- ***Checking if the Stack is Empty:***
  - **empty():** Returns 1 if the stack is empty, otherwise 0.

Quiz Time!

Key Takeaways

- Deque (Double-Ended Queue) **allows inserting**, **deleting**, and **accessing elements** from both ends efficiently.
- Stack follows **LIFO** (Last In, First Out), where the **last inserted** element is **removed first**.
- ***Operations***:
  - Deque: **push_front()**, **push_back()**, **pop_front()**, **pop_back()**, **insert()**, **erase()**.
  - Stack: **push()**, **pop()**, **top()**, **empty()**, **size()**.
- ***Usage***: Deques allow flexible element access at both ends, while **Stacks are useful** for managing sequential operations like function calls.

NXT WAVE™

THANK YOU

NXT WAVE™

ALL THE BEST

Iterating Over a Deque

C++

```
deque<int> d = {1, 3, 5, 7, 9};
for (auto it = d.begin(); it != d.end(); it++) {
  cout << *it << " ";
}
```

*Iterating using **Iterators**.*

Output

```
1 3 5 7 9
```

Inserting Elements at a Specific Position

C++

```
deque<int> d = {1, 3, 7, 9};
```

*C++ deque supports **insert()** with iterators*

Inserting Elements at a Specific Position

C++

```
deque<int> d = {1, 3, 7, 9};
auto it = next(d.begin(), 2);
```

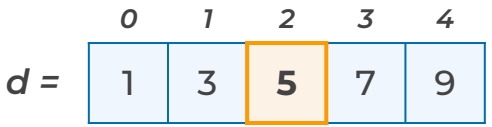
points iterator(it) to index 2

Inserting Elements at a Specific Position

C++

```
deque<int> d = {1, 3, 7, 9};
auto it = next(d.begin(), 2);
d.insert(it, 5);
```

Inserts 5 at index 2.



Removing Elements at a Specific Position

C++

```
deque<int> d = {1, 3, 7, 9};
```

*C++ deque supports **erase()** with iterators*

Removing Elements at a Specific Position

C++

```
deque<int> d = {1, 3, 7, 9};
auto it = next(d.begin(), 2);
```

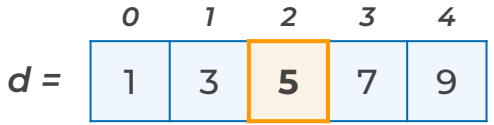
points iterator(it) to index 2

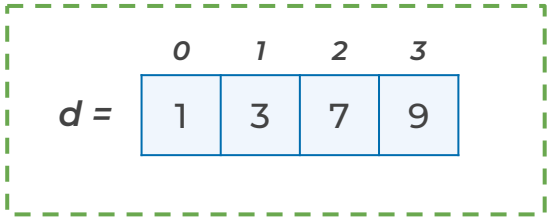
Removing Elements at a Specific Position

C++

```
deque<int> d = {1, 3, 7, 9};
auto it = next(d.begin(), 2);
d.erase(it);
```

Remove 5 at index 2.





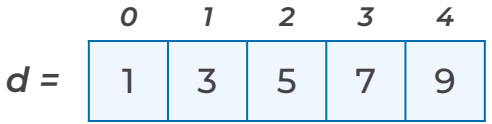
Removing Elements by Value

C++

```
deque<int> d = {1, 3, 5, 7, 9, 5};
```

*Deque of integers (**d**) initialized with*

*{**1**, **3**, **5**, **7**, **9**, **5**}*



Removing Elements by Value

C++

```
deque<int> d = {1, 3, 5, 7, 9, 5};
d.erase(remove(d.begin(), d.end(), 5), d.end());
```

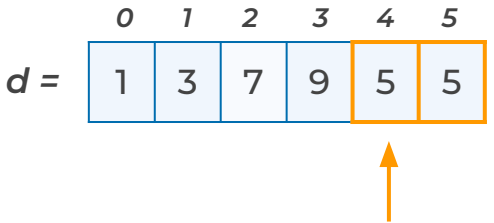
Removes occurrences of 5.

Deque

Removing Elements by Value

C++

```
deque<int> d = {1, 3, 5, 7, 9, 5};
d.erase(remove(d.begin(), d.end(), 5), d.end());
```



Moves all values not equal to 5 to the front and returns iterator to new logical end (first unwanted element).

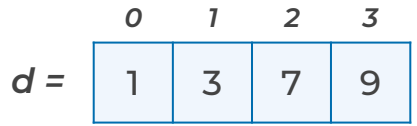
Removing Elements by Value

C++

```
deque<int> d = {1, 3, 5, 7, 9, 5};
d.erase(remove(d.begin(), d.end(), 5), d.end());
```

d.erase(remove(d.begin(), d.end(), 5), d.end())

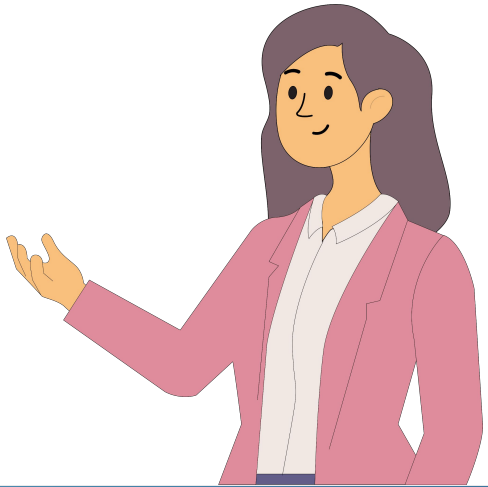
Erases all elements from new logical end to the actual end (removing all occurrences of 5).



Deque

Problem Statement

- You are tasked with developing an **order management system** for an **online food delivery service**.
- The system must efficiently handle customer orders using a **double-ended queue** (**deque**).
- Each order is identified by an **orderId**, and the system should support the following operations:



Deque

Supported Operations

The system will handle five types of operations:

- **AF** (**Add Order to Front**) – Adds a new order with the given orderId to the front of the deque.
- **AB** (**Add Order to Back**) – Adds a new order with the given orderId to the back of the deque.
- **RF** (**Remove Order from Front**) – Removes the order from the front of the deque.
- **RB** (**Remove Order from Back**) – Removes the order from the back of the deque.
- **D** (**Display Orders**) – Displays the current state of the deque.

Deque Operations

Example 1

Input

```
5
AF 101
AB 202
RF
AB 303
D
```

Output

```
202  303
```

- **AF 101**: Adds the order **101** to the front of the deque.
- **AB 202**: Adds the order **202** to the back of the deque.
- **RF**: Removes the order from the front, which is **101**.
- **AB 303**: Adds the order **303** to the back of the deque.
- **D**: Displays the current deque, which is [**202**, **303**]

Deque Operations

Solution

C++

```
void addFront(deque<int> &dq, int orderId) {
    dq.push_front(orderId);
}

void addBack(deque<int> &dq, int orderId) {
    dq.push_back(orderId);
}
```

Deque Operations

Solution

C++

```
void removeFront(deque<int> &dq) {
    if (!dq.empty()) {
        dq.pop_front();
    }
}
void removeBack(deque<int> &dq) {
    if (!dq.empty()) {
        dq.pop_back();
    }
}
```

Deque Operations

Solution

C++

```
void displayDeque(deque<int> &dq) {
  for (int order : dq) {
      cout << order << " ";
  }
  cout << endl;
}
```

Problem Statement

- Design a data structure called SpecialStack that supports the following operations:

push x : Adds an element x to the top of the stack.

pop : Removes and returns the top element from the stack.

getMin : Returns the minimum element currently present in the stack.

Special Stack

Example

Input:

push 5

getMin

push 3

getMin

pop

getMin

Special Stack

Example

Explanation:

push 5

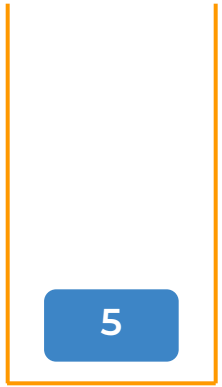
getMin

push 3

getMin

pop

getMin



Stack

Push “5” into Stack

Special Stack

Example

Explanation:

push 5

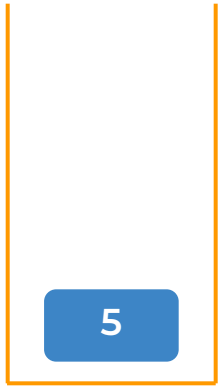
getMin

push 3

getMin

pop

getMin



Stack

Minimum element is “5”

Special Stack

Example

Explanation:

push 5

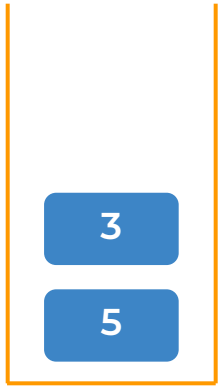
getMin

push 3

getMin

pop

getMin



Stack

Push “3” into Stack

Special Stack

Example

Explanation:

push 5

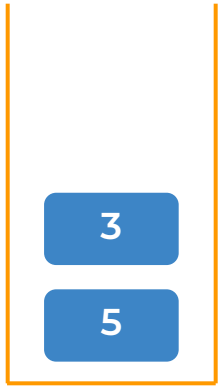
getMin

push 3

getMin

pop

getMin



Stack

Minimum element is “3”

Special Stack

Example

Explanation:

push 5

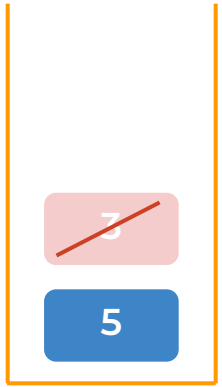
getMin

push 3

getMin

pop

getMin



Stack

Removes “3”.

Special Stack

Example

Explanation:

push 5

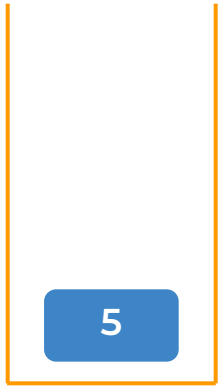
getMin

push 3

getMin

pop

getMin



Stack

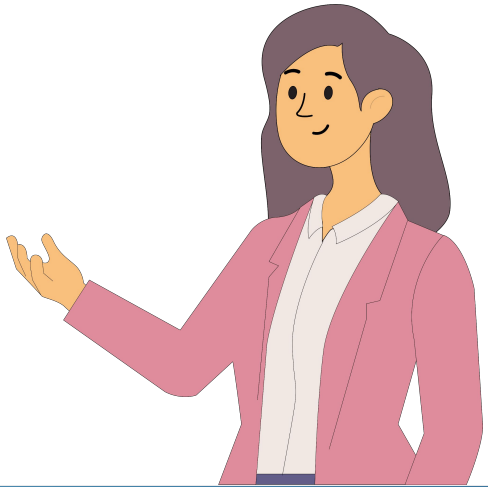
Minimum element is “5”

Code

Code

```
#include <bits/stdc++.h>
using namespace std;
class SpecialStack {
public:
   stack<int> mainStack;
   stack<int> minStack;
   void push(int x) {
       mainStack.push(x);
       if (minStack.empty() || x <= minStack.top()) {
           minStack.push(x);
       }
   }
```

continued

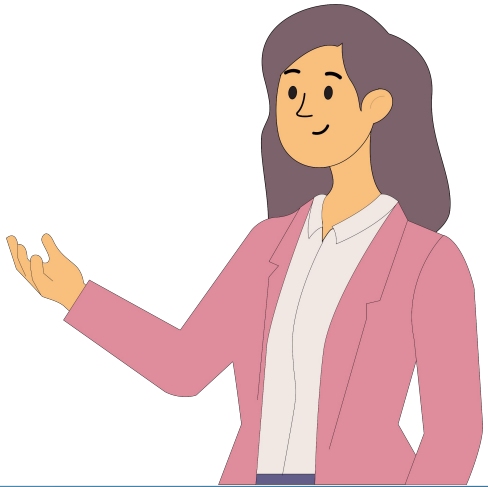


Code

Code

```
int pop() {
    if (mainStack.empty()) {
        cout << "Stack is empty" << endl;
        return -1;
    }
    int topElement = mainStack.top();
    mainStack.pop();
    if (topElement == minStack.top()) {
        minStack.pop();
    }
    return topElement;
}
```

continued



Special Stack

Code

Code

```
    int getMin() {
        if (minStack.empty()) {
            return -1;
        }
        return minStack.top();
    }
    bool isEmpty() {
        return mainStack.empty();
    }
};
```